本节内容：

- 变量
- 数据类型
- 类型转换

一、JavaScript介绍

结构：HTML（决定了身体）　表现：CSS（决定了样式美观）　行为：JavaScript（决定了交互的动态效果）

JavaScript（简称“JS”）是当前最流行、应用最广泛的客户端脚本语言，用来在网页中添加一些动态效果

1.1 书写位置

1. 内部JavaScript

直接写在html文件里，用script标签包住

规范：script标签写在 `</body>` 上面

拓展： alert(‘你好，js’) 页面弹出警告对话框

```
<body>
    <script>
        alert("这是一个弹窗！")
    </script>
</body>
```

2. 外部JavaScript

代码写在以.js结尾的文件里

语法：通过script标签，引入到html页面中。

```
<body>
    <!-- 通过src引入外部js文件 -->
    <script src="my.js"></script>
</body>
```

1.2 注释

- 单行注释

- **符号：** //
- **作用：** //右边这一行的代码会被忽略
- **快捷键：** ctrl + /

```
<script>
  // 这种是单行注释的语法
  // 一次只能注释一行
  // 可以重复注释
</script>
```

- 块注释
  - **符号：** /* */
  - **作用：** 在/* 和 */ 之间的所有内容都会被忽略
  - **快捷键：** shift + alt + A

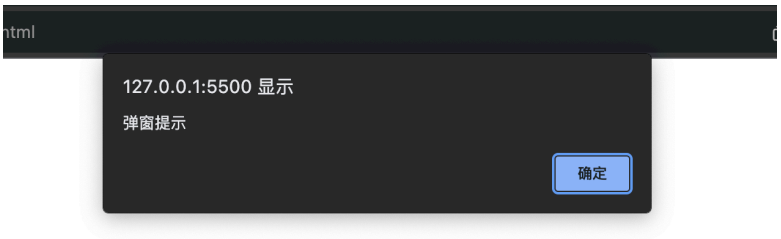
```
<script>
  /* 这种的是多行注释的语法 */
  /*
    更常见的多行注释是这种写法
    在些可以任意换行
    多少行都可以
 */
</script>
```

1.3 输出

1. **弹窗**

- **函数：** `alert()`
- **作用：** 页面弹出警告对话框
- **代码：**

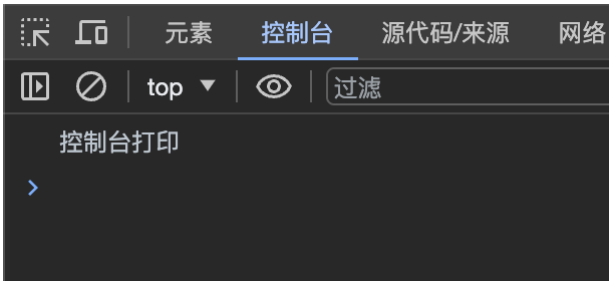
```
alert('弹窗提示')
```



2. 控制台打印

- **函数：** `console.log()`
- **作用：** 控制台输出，程序员调试使用
- **代码：**

```
console.log('控制台打印')
```



1.4 JavaScript 结束符

作用： 使用英文的 ; 代表语句结束

实际情况： 实际开发中，可写可不写, 浏览器 (JavaScript 引擎) 可以自动推断语句的结束位置

现状： 在实际开发中，越来越多的人主张，书写 JavaScript 代码时省略结束符

约定： 为了风格统一，结束符要么每句都写，要么每句都不写（按照团队要求.）

二、变量

介绍： 在JavaScript中定义变量需要提前声明

步骤：

1. 声明变量
2. 赋值

3. 使用变量

2.1 let 关键字

作用： 声明一个变量

代码：

```
// 方式一：
// 声明变量
let name
// 对变量赋值
name = '张三'

// 方式二：
// 声明变量和赋值同时进行
let name = '张三'
```

> **注意：**
>
> 1. 使用let声明的变量不可重复声明

扩展：

let和var的区别：

在较旧的JavaScript，使用关键字 var 来声明变量，而不是 let。var 现在开发中一般不再使用它，只是我们可能再老版程序中看到它。let 为了解决 var 的一些问题。

var 声明:

- 可以先使用 在声明 (不合理)
- var 声明过的变量可以重复声明(不合理)
- 比如变量提升、全局变量、没有块级作用域等等

结论：

`var 就是个bug，别迷恋它了，以后声明变量我们统一使用 let`

2.2 const 关键字

使用场景： 当某个变量永远`不会改变`的时候，就可以使用 const 来声明，而不是let。

命令规范： 和变量一致

代码：

```
// 定义一个常量
const id = 10
// 输出这个常量
console.log(id)
```

> **注意：**
>
> 1. 常量不允许被重复声明
> 2. 常量不可被修改

2.3 小结

- let： 现在实际开发变量声明方式
- var：以前的声明变量的方式，会有很多问题
- const：类似于let，但是变量的值无法被修改

三、数据类型

JavaScript 的数据类型分为两大类：**原始类型（Primitive）** 和 **对象类型（Object）**

3.1 原始类型

原始类型是 JavaScript 中最基础的数据类型，它们是不可变的（immutable），直接存储在栈内存中

原始数据类型：

- Number（数字型）
- String（字符串型）
- Boolean（布尔型）
- Undefined（未定义型）
- Null（空类型）
- BigInt(大整数)
- Symbol（符号）

3.1.1 Number（数字）

介绍： 即我们数学中学习到的数字，可以是整数、小数、正数、负数。

```
let age = 18   // 整数
let price = 88.99 // 小数
```

数字可以有很多操作，比如，乘法 * 、除法 / 、加法 + 、减法 - 等等，所以经常和算术运算符一起。

数学运算符也叫**算术运算符**，主要包括加、减、乘、除、取余（求模）。

- +：求和
- -：求差
- *：求积
- /：求商
- %：取模（取余数）

3.1.2 String（字符串）

介绍： 通过单引号 `''` 、双引号 `""` 或反引号 ` 包裹的数据都叫字符串，单引号和双引号没有本质上的区别，推荐使用单引号。

```
let uname = '小明' // 使用单引号
let gender = "男"  // 使用双引号
let goods = `小米`  // 使用反引号
let tel = '13681113456'  // 看上去是数字，但是引号包裹了就是字符串
let str = '' // 这种情况叫空字符串
```

- **模板字符串**
- **使用场景：**
- 拼接字符串和变量
- 在没有它之前，要拼接变量比较麻烦

```
document.write('大家好，我叫' + name + '，今年' + age + '岁')
```

- **语法**
- `` (反引号)
- 在英文输入模式下按键盘的tab键上方那个键（1左边那个键）
- 内容拼接变量时，用 ${ } 包住变量

```
document.write(`大家好，我叫${name}，今年${age}岁`)
```

3.1.3 Boolean（布尔值）

表示肯定或否定时在计算机中对应的是布尔类型数据。

它有两个固定的值 true 和 false，表示肯定的数据用 true（真），表示否定的数据用 false（假）。

```
// JavaScript 好玩不?
let isCool = true
console.log(isCool)
```

3.1.4 Undefined（未定义）

介绍： 未定义是比较特殊的类型，只有一个值Undefined。

什么情况出现未定义类型?

只声明变量，不赋值的情况下，变量的默认值为Undefined。

```
let name
console.log(name);  // undefined
```

3.1.5 Null（空值）

介绍： JavaScript 中的 Null 仅仅是一个代表“无”、“空”或“值未知”的特殊值

Null 和 Undefined 区别：

- Undefined 表示没有赋值
- Null 表示赋值了，但是内容为空

Null 开发中的使用场景：

官方解释：把 Null 作为尚未创建的对象

大白话： 将来有个变量里面存放的是一个对象，但是对象还没创建好，可以先给个null

3.2 检测数据类型

通过 typeof 关键字检测数据类型

typeof 运算符可以返回被检测的数据类型。它支持两种语法形式：

1. 作为运算符： typeof x （常用的写法）
2. 函数形式： typeof(x)

换言之，有括号和没有括号，得到的结果是一样的，所以我们直接使用运算符的写法

```
let age = 18
let uname = '刘德华'
let flag = false
let buy
console.log(typeof age)
console.log(typeof uname)
console.log(typeof flag)
console.log(typeof buy)
```

3.3 类型转换

3.3.1 隐式转换

某些运算符被执行时，系统内部自动将数据类型进行转换，这种转换称为隐式转换。

规则：

- +号两边只要有一个是字符串，都会把另外一个转成字符串
- 除了+以外的算术运算符 比如 - * / 等都会把数据转成数字类型

缺点：

- 转换类型不明确，靠经验才能总结

小技巧：

- 任何数据和字符串相加结果都是字符串

```
<script>
    console.log(11 + 11)
    console.log('11' + 11)
    console.log(11 - 11)
    console.log('11' - 11)
    console.log(1 * 1)
    console.log('1' * 1)
    console.log(typeof '123')
    console.log(typeof +'123')
    console.log(+'11' + 11)
</script>
```

```
22
1111
0
0
1
1
string
number
22
```

3.3.2 显示转换

编写程序时过度依靠系统内部的隐式转换是不严禁的，因为隐式转换规律并不清晰，大多是靠经验总结的规律。

为了避免因隐式转换带来的问题，通常根逻辑需要对数据进行显示转换。

概念：

自己写代码告诉系统该转成什么类型

转换为数字型

- Number(数据)
  - 转成数字类型
  - 如果字符串内容里有非数字，转换失败时结果为 NaN（Not a Number）即不是一个数字

    - NaN也是number类型的数据，代表非数字
- parseInt(数据)
    - 只保留整数
- parseFloat(数据)
    - 可以保留小数

转换为字符型:

- String(数据)

3.4 对象类型

- **Object（对象）**
- **Array（数组）**
- **Function（函数）**
- ...

3.4.1 数组

声明语法:

```
let ls = [数据1, 数据2, 数据3, ...]
```

例如:

```
let ls = ['张三', '李四', '王五']
```

3.4.2 数组的常见操作

1. 访问数组元素:

```
let arr = [1, 2, 3];
console.log(arr[0]);  // 输出: 1
console.log(arr[1]);  // 输出: 2
console.log(arr[2]);  // 输出: 3
```

2. 修改数组元素:

```
let arr = [1, 2, 3];
arr[0] = 4;  // 修改数组中的第一个元素
console.log(arr);  // 输出: [4, 2, 3]
```

3. 获取数组长度:

```
let arr = [1, 2, 3];
console.log(arr.length);  // 输出: 3
```

4. 添加新元素到数组末尾:

```
let arr = [1, 2, 3];
arr.push(4);
console.log(arr);  // 输出：[1, 2, 3, 4]
```

5. 添加元素到指定位置 `splice(start, deleteCount=0, ...items)`

   参数：

   - `start` （起始索引）
   - `deleteCount=0` （删除数量）
   - `...items` （插入的新元素）

```
let arr = [1, 2, 3];
arr.splice(1, 0, 'b');
```

6. 删除指定位置开始的元素 `splice(start, deleteCount=0, ...items)`

   参数：

   - `start` （起始索引）
   - `deleteCount=0` （删除数量）
   - `...items` （省略第三个元素）

```
let arr = [1, 2, 3];
arr.splice(1, 2);
```